ONTARIO TEJ4M COMPUTER ENGINEERING TECHNOLOGY

HARDWARE DESIGN DOCUMENT

PROJECT

Bland2Grand

Smart Autonomous Carousel Spice Dispensing Appliance

Student	Elliott Starosta
Course	TEJ4M Computer Engineering Technology
Teacher	Mr. Roller
Version	1.0
Date	June 19, 2026

Contents

1 Introduction

1.1 Project Overview

Bland2Grand is a smart, autonomous countertop spice dispensing appliance that eliminates the guesswork from seasoning food. The primary audience is home cooks who want precise, repeatable seasoning without measuring spoons — particularly those cooking for varying portion counts, experimenting with new cuisines, or lacking familiarity with the spices and flavor combinations commonly used in different culinary traditions. A user types the name of any dish into a mobile web application, the system retrieves matching spice blend options from a local recipe database, the user selects a blend and a serving count, and the machine autonomously dispenses the correct gram-accurate quantity of each required spice directly into a bowl placed below — with no manual measurement required at any step.

The physical machine is centred on a motorised carousel. Eight interchangeable spice containers are seated one per bay around the carousel ring. The carousel is driven by a NEMA 23 stepper motor (M1) through a 9:1 cycloidal gearbox for smooth, high-torque indexing. Once the correct container is indexed to the active position, a NEMA 17 stepper motor (M2) fixed to the tower frame below the carousel plane engages the container's internal auger screw through a bevel gear interface, pushing spice through a downward-facing nozzle into the bowl below. A NOYITO 1 kg load cell (LC1) paired with an HX711 24-bit ADC amplifier (U3) provides real-time gram-accurate weight feedback. An Arduino Uno R4 WiFi (U1) coordinates all motion, sensing, and wireless communication with a Flask web server running on a host laptop over the onboard 2.4 GHz 802.11 b/g/n WiFi module.

This Hardware Design Document (HDD) is the final capstone document for the Bland2Grand project. It documents *what was actually built*, how it differs from the Preliminary Design Document (PDD), how to use the system, and reflects on the overall learning process.

1.2 Related Documents

- **Hardware Requirements Specification (HRS)**: The authoritative design contract defining all *shall* requirements for the system, authored March 8, 2026.
- **Preliminary Design Document (PDD)**: The design document describing how the HRS requirements would be met, authored March 31, 2026.
- **Development Journals (1–10)**: Weekly build logs documenting the full design and build process from March to June 2026.

1.3 Firmware, Platforms, and APIs

The firmware runs on the Arduino Uno R4 WiFi using the Arduino framework via PlatformIO. The following open-source libraries were imported:

Library	Purpose
AccelStepper v1.64	Non-blocking step generation for M1 and M2
bogde/HX711 v0.7.5	Load cell ADC reading; confirmed Uno R4 WiFi-compatible
robtillaart/AS5600 v0.6.7	I2C encoder readout; imported but unused in final build
bblanchon/ArduinoJson v7.2.2	JSON serialisation and deserialisation over WiFi
jandrassy/ArduinoOTA v1.1.1	Over-the-air firmware update support

The Arduino firmware was written entirely in C++ and is structured as a flat state machine in `main.cpp` cycling through `IDLE` → `INDEXING` → `DISPENSING` → `PARKING`. Four subsystem headers encapsulate all hardware logic: `CarouselDriver.h` handles CCW slot indexing with a per-slot step correction constant; a back-purge reversal step provides passive anti-drip behaviour (see Section 2.1.3 for full rationale); per-spice timeout and watchdog behaviour are listed in Section 3.6; `Scale.h` wraps the HX711 with EMA filtering, a blocking stable-read routine, and overload detection; and `WiFiComms.h` manages WiFi connection, a minimal HTTP server on port 80, and outbound UDP weight-update packets on port 5001 (see Section 2.2 for the full network architecture and protocol rationale). `FlowModel.h` provides EEPROM-backed per-slot linear regression and coast estimation. A `generate_slots.py` script at the repository root regenerates `SlotConfig.h`, `slot_config.py`, and `slotConfig.ts` from a single `spice_slots.json` source of truth, propagating any spice slot change to all three layers simultaneously.

The Flask backend is Python 3 with Flask, Flask-CORS, requests, and python-dotenv. `app.py` defines all REST and SSE routes; `dispense.py` runs the dispense session on a background thread, fans SSE events out to all connected clients via a per-client `queue.Queue`, listens for Arduino weight pushes on UDP port 5001, and implements a `MOCK_ARDUINO` simulation mode that replays realistic dispensing behaviour in software so the frontend could be developed and tested without hardware. `search.py` implements a three-tier lookup: category keyword match, SQLite `LIKE` search, then OpenRouter AI fallback. `database.py` manages the SQLite schema and recipe CRUD, and `seed_recipes.py` populates the database with approximately 100 curated recipes on first run.

The frontend is React 18 / TypeScript built with Vite, styled with Tailwind CSS, and animated with GSAP. `App.tsx` owns screen routing via a custom `Screen` union type, a 60-second idle timer, and the dispense session lifecycle. `useDispenseStream.ts` implements the SSE hook: `connectAndDispense()` opens the event source, waits for the `connected` acknowledgement before POSTing the dispense request (guaranteeing no events are dropped on slow connections), then folds every subsequent event into session state via a pure reducer. `Bowl.tsx` renders an SVG bowl that fills with procedurally-grained, seeded-random spice layers as weight accumulates, with a GSAP drip animation on the active slot. `SlotRow.tsx` shows a per-spice animated progress bar driven by GSAP.

`IdleScreen.tsx` implements the screensaver with floating ambient orbs and a pulsing tap indicator. `CustomRecipeScreen.tsx` provides per-slot amount controls with a unit-picker bubble that animates open and closed with GSAP, converting between grams, teaspoons, and tablespoons using per-spice density constants.

The AI recipe generation uses the OpenRouter API (`anthropic/claude-3-haiku` by default) to generate JSON spice blends for dishes not found in the local SQLite database. API calls are made at most once per unique dish name; all subsequent requests are served from the local database cache.

1.4 Acknowledgements and AI Assistance

CAD (OnShape): Michael and Jiayuan taught me how to use OnShape from scratch, walked me through the cycloidal gearbox epitrochoid geometry, helped with the structural base redesign (gussets and perimeter lip), and printed parts on their home printers throughout the mechanical phase.

Logo design: Naomi designed the Bland2Grand octagonal B/G lettermark logo.

Firmware debugging: Claude (Anthropic) was used to help debug specific firmware issues during development, including the `EEPROM.commit()` requirement on the Renesas RA platform and resolving AccelStepper speed transition behaviour.

Development methodology: All design decisions and implementation work are original to Elliott Starosta. Claude was used only as a reference tool during specific debugging sessions and was never used to generate complete code files.

1.5 Onboard Storage and Memory Architecture

The Arduino Uno R4 WiFi's Renesas RA4M1 provides 256 KB of flash for program storage and 32 KB of SRAM for runtime variables, along with 8 KB of data flash used for EEPROM emulation to store non-volatile data across power cycles. `FlowModel.h` writes the learned grams-per-revolution constant for each of the 8 slots to EEPROM after every dispense, so the system's calibration improves across power cycles rather than resetting to a default estimate every time the machine is unplugged. This is the only persistent storage in the system; recipe and session data live entirely on the host laptop's SQLite database rather than onboard, since the Arduino's limited flash and RAM are reserved for firmware logic and motor control rather than data storage. This division reflects a deliberate hardware tradeoff: the embedded controller is sized for real-time control, not data persistence, so all higher-volume storage (recipes, AI-generated blends, session history) is offloaded to the host machine's conventional storage rather than burdening the microcontroller's limited onboard memory.

One notable hardware quirk encountered during development was that the Renesas RA4M1's `EEPROM.commit()` requirement differs from the byte-addressable EEPROM behaviour of older AVR-based Arduino boards (e.g. the Uno R3's ATmega328P), where writes are immediate. On the RA4M1, EEPROM is emulated in flash, and writes must be explicitly committed in a single batch operation or they are silently discarded on reset. This is a direct example of how advances in microcontroller architecture (moving from

native EEPROM to flash-emulated EEPROM for higher storage density) change low-level programming assumptions that carry over from older hardware generations.

2 Updated System Description

2.1 What Changed from the PDD

Several significant design decisions changed during the build phase. All changes are documented here as deviations from the PDD, with rationale.

2.1.1 Carousel Drive: Cycloidal Gearbox Replaces Pinion-and-Ring

PDD design: A pinion gear on M1's shaft meshing externally with a ring gear on the carousel platform outer rim (GR = 2).

Actual design: A 9:1 cycloidal gearbox sits between M1's shaft and the carousel center hub. The cycloidal disc (9 lobes) rolls inside a ring of 10 pins, with the output taken off through an output carrier.

Rationale: During physical assembly, it became clear that the external spur mesh was sensitive to center-distance variation under real rotating load. With eight loaded containers on the platform, any tilt of the carousel disc changes the mesh depth dynamically and causes skipped teeth and position errors. The cycloidal design distributes the contact load across multiple pins simultaneously (roughly half the pins are in contact at any time), is inherently self-centering, and has dramatically lower backlash. The 9:1 reduction also provides more torque to the carousel than the original 2:1 ratio. Michael identified this problem and recommended the change during the assembly phase.

2.1.2 AS5600 Encoder: Removed

PDD design: An AS5600 12-bit absolute magnetic encoder mounted coaxially on the M1 shaft, providing closed-loop angular position verification after every carousel index move.

Actual design: No encoder is fitted. The carousel operates entirely open-loop, relying on the AccelStepper step count to determine position.

Rationale: The AS5600 module purchased was listed by the supplier as compatible with the NEMA 23 motor shaft geometry. On arrival, the module's magnet holder could not be made to maintain the required 0.5 mm axial air gap on the actual motor shaft diameter and shoulder geometry. Several mounting approaches were attempted but none produced stable angle readings. With insufficient time remaining in the build schedule to source a replacement encoder or design a new magnet holder, the encoder was removed from the design. As a consequence, HRS-3.1.21, HRS-3.1.22, and HRS-3.1.23 are not met in the final build. The carousel homing routine described in the PDD was also removed; on power-up the firmware assumes the carousel is at slot 1 and the user must manually verify this before the first dispense.

2.1.3 Auger Engagement: Bevel Gears Replace Half-Spur-Gear

PDD design: A half spur gear on M2's shaft (teeth on 180° only) engaging a full spur gear on each container's top face, with the toothless arc providing a positive mechanical cutoff.

Actual design: A bevel gear on M2's shaft engages a matching bevel gear on each container's auger shaft. The bevel gear pair transmits rotation through 90°. A back-purge routine in the firmware reverses the auger by the same number of steps taken during the dispense, sweeping residual powder back up the helix and creating a passive anti-drip condition.

Rationale: The half-spur gear concept was sound in CAD but the PDD's spring-loaded jaw coupler variant created complex axial alignment requirements. The bevel gear approach that emerged during actual build is mechanically simpler: the motor gear and container gear are always in the same plane, engagement happens passively as the carousel rotates, and the back-purge provides the anti-drip function previously served by parking the toothless arc. The firmware now tracks total steps per dispense and issues an equal reverse move after each dispense completes.

2.1.4 Logic Level Shifting: Removed (Carried Forward from PDD)

As documented in the PDD Design Revision Note, HRS-3.3.1.1, HRS-3.3.1.2, and HRS-3.3.1.7 are superseded. The Funsto TB6600 units used for U4 and U5 specify a control signal input range of 3.3–24 V, which directly accommodates the RA4M1 output. No level shifter was installed.

2.1.5 Power Switch: Not Installed

PDD/HRS design (HRS-3.1.38): A panel-mounted SPST rocker switch on the 24 V line, providing a single hardware-level power cut to the entire system.

Actual design: No rocker switch was installed. To power the machine on and off, the 24 V supply is plugged into a power bar with its own on/off switch, which serves the same function.

Rationale: The switch cutout was not added to the enclosure before the final assembly was closed up, and there was insufficient time to reprint the affected panel. The power bar workaround is functionally equivalent for demonstration purposes (see HRS-3.1.38).

2.1.6 Power Supply Buck Converter Output

PDD design: LM2596S-ADJ trimmed to 5.0 V.

Actual design: Trimmed to 7.1 V, feeding the Arduino's VIN pin rather than the 5 V pin, taking advantage of the RA4M1's onboard voltage regulator. Testing results are documented in HRS-3.1.36.

2.1.7 Auger Tube Clearance

PDD design: 0.2 mm radial clearance between auger helix OD and tube ID.

Actual design: Increased to 0.3 mm per side (0.6 mm total diametric clearance) after

empirical testing revealed the original clearance caused powder jamming with fine spices such as salt and cumin. The additional 0.1 mm per side was required to keep the auger-to-tube clearance ratio within the 5–15% range recommended for powder conveyors.

2.2 Network Architecture and Addressing

The system runs as a private local network created by the host laptop's Windows mobile hotspot rather than relying on existing home WiFi infrastructure, so the demo is portable and does not depend on a venue's network configuration. The laptop's hotspot acts as a DHCP server, assigning IP addresses to two clients: the Arduino Uno R4 WiFi (firmware joins as a WiFi station and is assigned an address dynamically) and any phone running the web app. Because addresses are assigned dynamically rather than statically, the Arduino's IP is read off the serial monitor at boot rather than hardcoded, and the Flask backend's listening address is likewise read from the Vite terminal output rather than assumed; the User's Guide instructs the user to confirm both devices report addresses on the same subnet before attempting to connect, which is the practical addressing check needed on a small DHCP-assigned network with no router-level configuration access.

Three distinct ports are used for three distinct communication purposes, a separation that exists specifically to prevent one type of traffic from blocking another: port 80 on the Arduino accepts inbound HTTP POST requests from Flask containing dispense commands; port 5000 on the laptop is where Flask's REST API (`/search`, `/dispense`, `/status`, `/calibrate`, `/recipe`) is exposed to the phone's browser; and port 5001 carries outbound UDP weight-update packets from the Arduino to Flask. UDP was chosen deliberately for the weight stream rather than HTTP, because UDP has no connection handshake and no retransmission guarantee, so a dropped or delayed packet on a momentarily congested hotspot cannot stall the Arduino's stepper-control loop the way a blocking TCP handshake could. This represents a real network design tradeoff between reliability and the embedded controller's real-time obligations: weight updates are frequent and individually low-stakes (the next one will arrive in well under a second if one is lost), so the lower-overhead, unreliable transport is the better fit, whereas dispense commands and final status reports are sent over the reliable, connection-oriented HTTP path, since a lost command would actually break the system.

2.3 Updated System Block Diagram

The block diagram below reflects the final as-built system architecture.

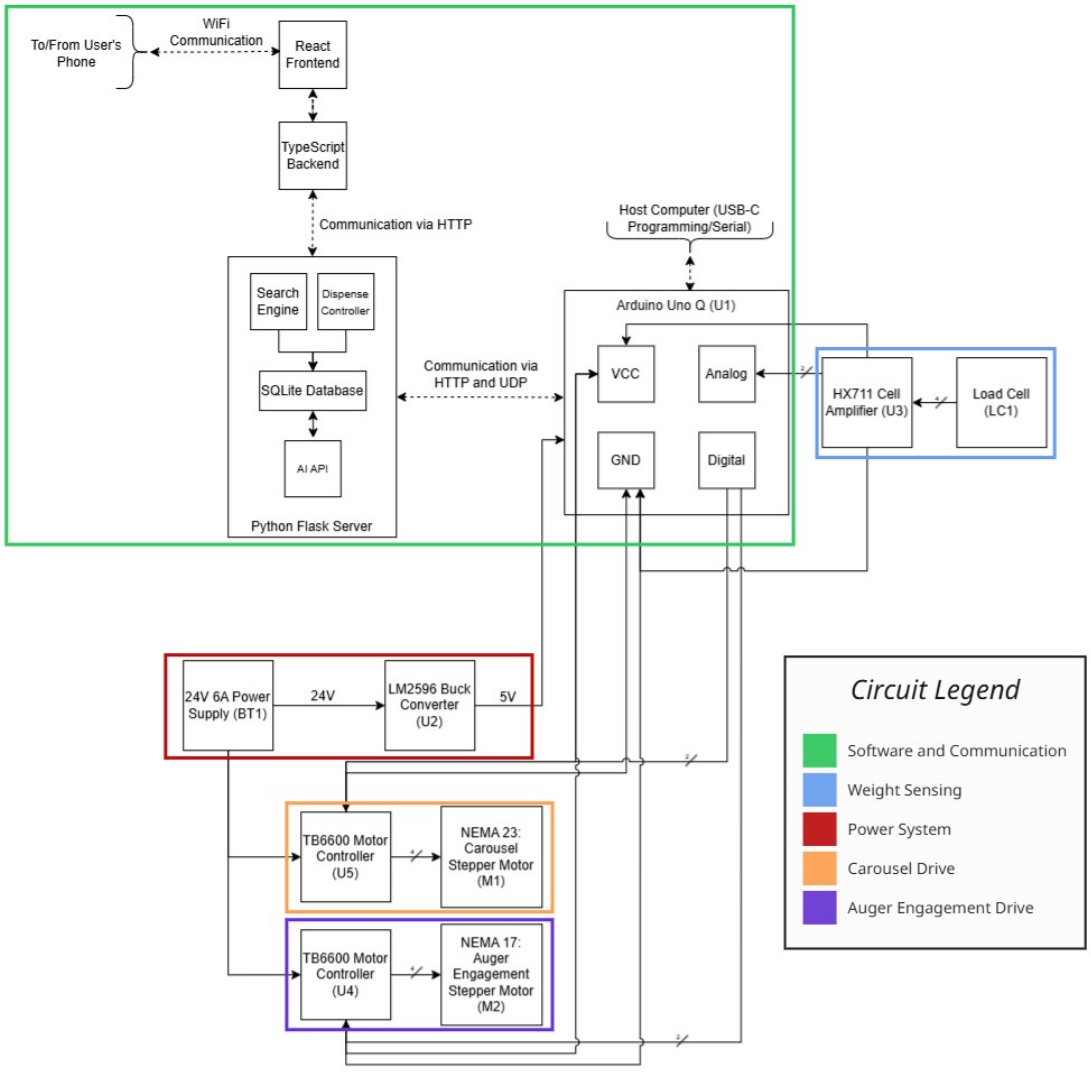


Figure 1: Updated as-built system block diagram.

3 User's Guide

3.1 System Overview for the End User

Bland2Grand is a countertop appliance that automatically dispenses the right spice blend for any dish. You interact with it entirely through a web app on your phone. The machine handles all indexing, dispensing, and weight confirmation autonomously and reports the result back to your phone in real time.

Figure 2: Bland2Grand assembled system.

3.2 Hardware Setup

1. **Place the machine on a flat, level surface.** The load cell platform must remain level to within 2 degrees for accurate readings. Avoid placing on uneven countertops or surfaces that vibrate.

2. **Connect the 24 V power supply.** Plug the 24V power input into the wall.

3. **Verify spice containers are loaded.** Each of the 8 slots should contain its designated spice (see slot map in Section 3.5). Friction-fit lids pull straight off; do not twist.

4. **Place an empty bowl on the scale platform.** The bowl should sit flat and centred on the platform. The scale will tare before each spice dispense, so the exact bowl weight does not matter.

Figure 3: Bowl placed on the scale platform at the base of the machine.

5. **Ensure the host laptop is on and connected to the hotspot.** The Arduino connects to a mobile hotspot created by the laptop (see Section 3.3). The phone that will run the app must be on the same network.

3.3 Launching the Software

All software is launched from VSCode using the provided `tasks.json`. Open the project workspace in VSCode, then open the Command Palette with **Ctrl+Shift+P** and select **Tasks: Run Task**.

Figure 4: VSCode Command Palette showing “Tasks: Run Task” selected.

Two compound tasks are available:

Task Name	What it does
Bland2Grand: Full Stack	Flashes firmware to the Arduino, opens the serial monitor, starts the Flask backend, and starts the Vite frontend — all in parallel. Use this for a full bring-up from scratch.
Bland2Grand: Backend + Frontend	Starts only the Flask backend and the Vite frontend. Use this when the Arduino is already flashed and running. This is the task to run for a demo.

What Full Stack does automatically

The **Arduino: Flash + Serial** subtask does three things before flashing: it starts the Windows mobile hotspot (so the Arduino has a network to connect to), kills any open serial monitor that would block the port, then runs `pio run --target upload` followed by `pio device monitor --baud 9600`. No manual hotspot setup or port management is needed.

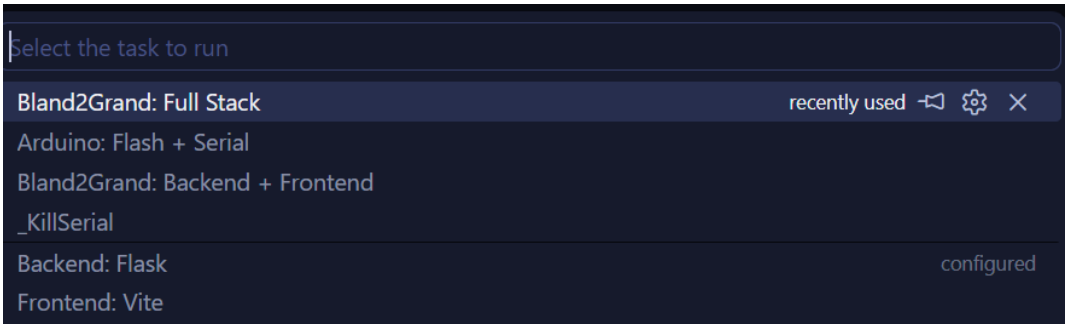


Figure 5: VSCode task picker showing the available Bland2Grand tasks.

Once the **Backend + Frontend** task is running, the web app is accessible from any device on the same network at:

$$\texttt{http://\{laptop\_ip\}:5173}$$

The laptop's IP address is printed in the Vite terminal panel when it starts. On the Arduino's serial monitor you will also see the IP it received from the hotspot, which confirms the two are on the same subnet.

```
> bland2grand-frontend@0.1.0 dev
> vite


  VITE v5.4.21  ready in 1122 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://192.168.137.1:5173/
  ➜  Network: http://192.168.2.71:5173/
  ➜  Network: http://172.31.64.1:5173/
```

Figure 6: Vite dev server terminal output showing the network URL to open on the phone.

3.4 Using the Mobile Web App

3.4.1 Idle Screen — Tap to Wake

When you first open the URL on your phone, the idle screen is shown: ambient coloured orbs drift across a dark background with the Bland2Grand logo and a pulsing tap indicator. Tap anywhere on the screen to wake the app.

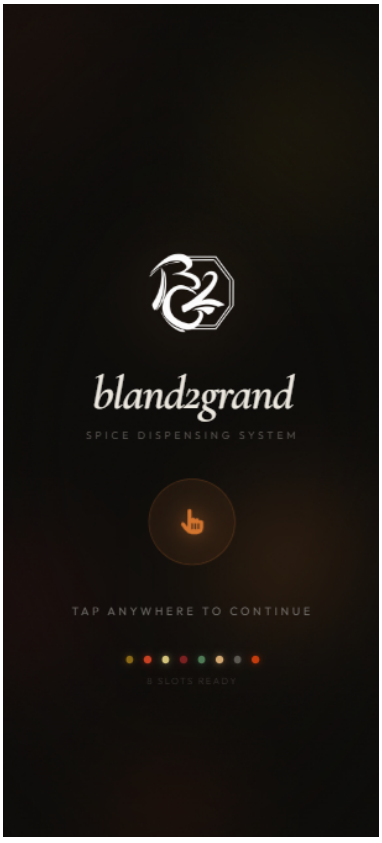


Figure 7: Idle screen. Tap anywhere to continue.

3.4.2 Search Screen — Find a Recipe

The search screen has three ways to find a recipe:

1. **Type a dish name** into the search bar. Results appear automatically after a 2.5-second pause. If no local match exists, the AI generates a blend and saves it to the database for next time.
2. **Tap a Featured Blend** tile to jump directly to a popular recipe without searching.
3. **Tap a cuisine category** (Mexican, Indian, Italian, etc.) to browse all recipes in that category.

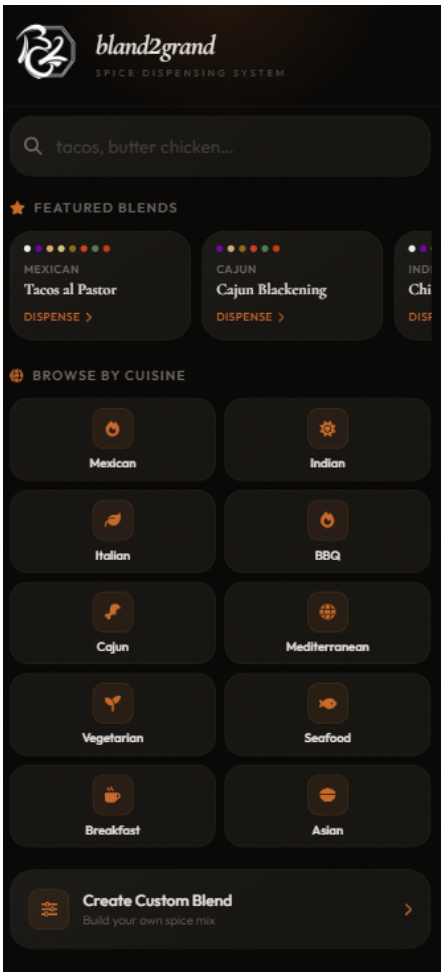
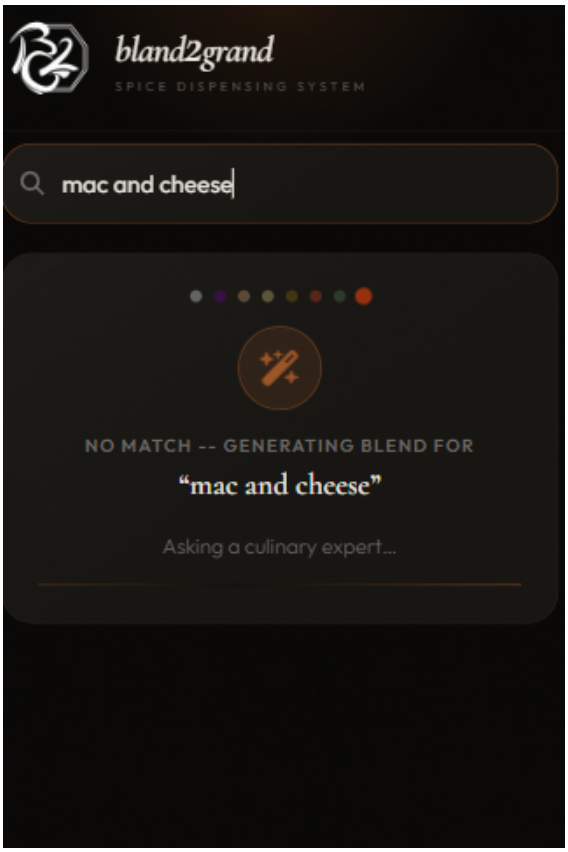


Figure 8: Search screen showing the search bar, featured blends, and cuisine category grid.

If the dish is not in the local database, an AI loading overlay appears while the blend is being generated. This takes 1–3 seconds and only happens once per unique dish name — all future searches for the same dish are served instantly from the cache.



AI Loading

Generated Result

Figure 9: AI loading overlay shown when generating a new recipe blend and the resulting generated spice blend.

3.4.3 Results Screen — Choose Your Blend

Up to six matching recipes are shown as cards. Each card displays the recipe name, cuisine category badge, a proportional spice colour bar, and the gram amount per serving for each active spice. Tap a card to select it.

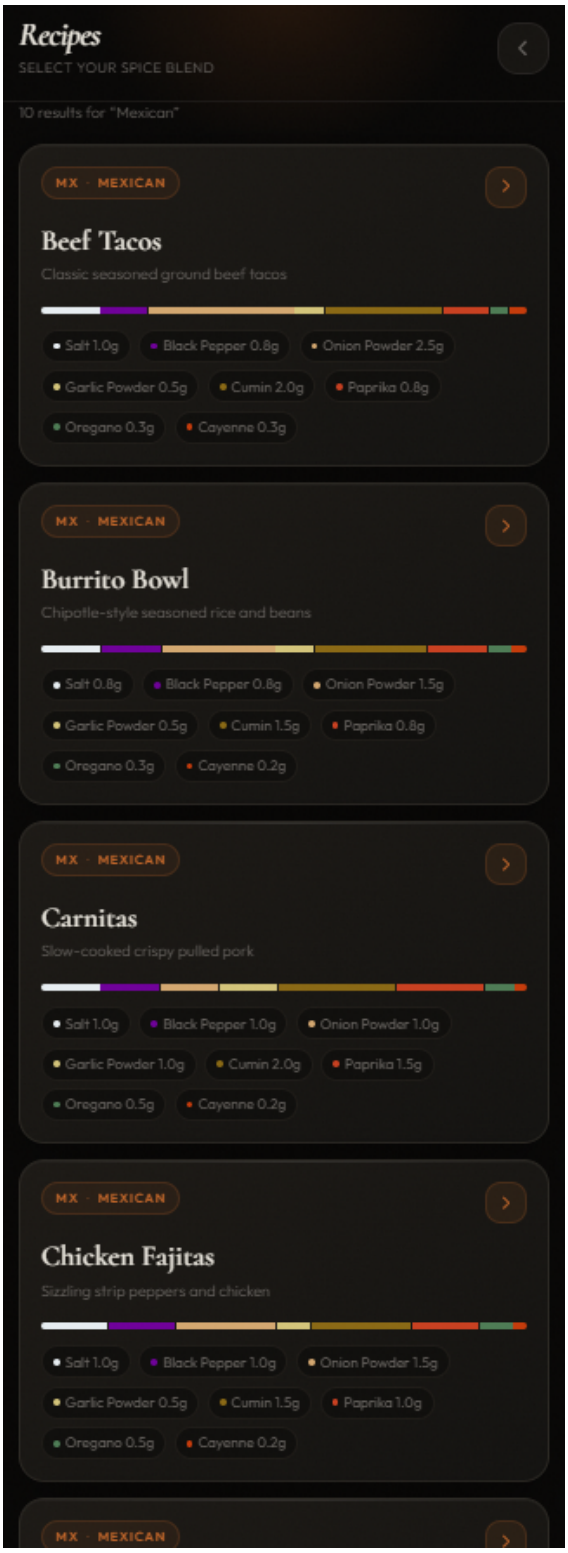


Figure 10: Results screen showing matching recipe cards with spice colour bars and gram breakdowns.

3.4.4 Serving Screen — Set Portions

Use the large **+** and **−** buttons or the quick-pick row (1, 2, 4, 6, 8) to set how many servings to dispense. All gram amounts update in real time as you adjust. When ready, tap **Dispense N servings**.

Figure 11: Serving screen with the counter set to 1 servings and gram breakdown updating live.

3.4.5 Dispensing Screen — Watch It Work

The dispensing screen shows:

- A **bowl visualisation** that fills with layered spice in each slot's colour as weight accumulates.
- A **per-spice progress row** with a live bar, status icon (rotating for indexing, spinning for dispensing, checkmark for done), and current vs. target weight.
- A **circular progress ring** in the top-right showing overall completion percentage.
- A **status line** showing the current action (e.g., *"Rotating. . . "* or *"Cumin"*).

Each spice plays a voice announcement when the carousel arrives at its slot. Do not remove the bowl during dispensing.

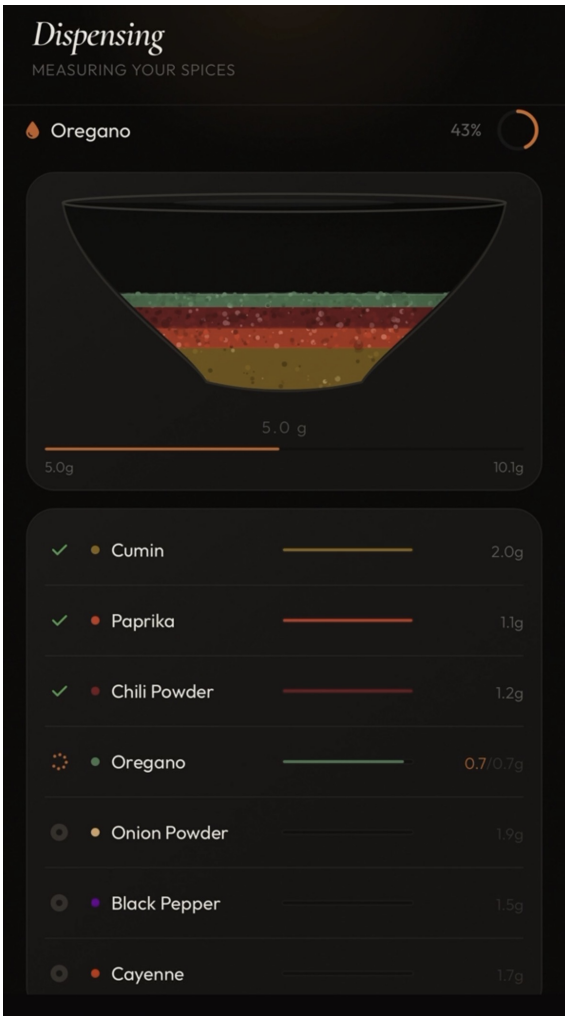


Figure 12: Dispensing screen mid-session: bowl filling, per-spice rows, and progress ring.

3.4.6 Complete Screen — Your Blend Is Ready

When all spices have been dispensed, the screen transitions to the Ready screen. It shows:

- A green checkmark (or amber warning if any slot timed out).
- A summary card listing the actual dispensed weight for each spice alongside its target. Spices within 0.5 g of target are shown in green.
- The total dispensed weight.

Retrieve your bowl and start cooking. Tap **Start a new blend** to return to the search screen.

Figure 13: Complete screen showing the dispensed summary with accuracy indicators.

3.4.7 Custom Recipe Builder

Tap **Create Custom Blend** on the search screen to build your own blend from scratch.

1. Enter a blend name and optional description in the fields at the top.

2. For each of the 8 slots, tap the unit label (`tsp`, `tbsp`, or `g`) to cycle between units. Amounts convert automatically using the per-spice density constant.

3. Use the **+** and − buttons to set the amount for each spice. A live proportional bar chart at the top updates as you adjust.

4. Tap **Save & Dispense** to write the recipe to the database and proceed to the serving screen.

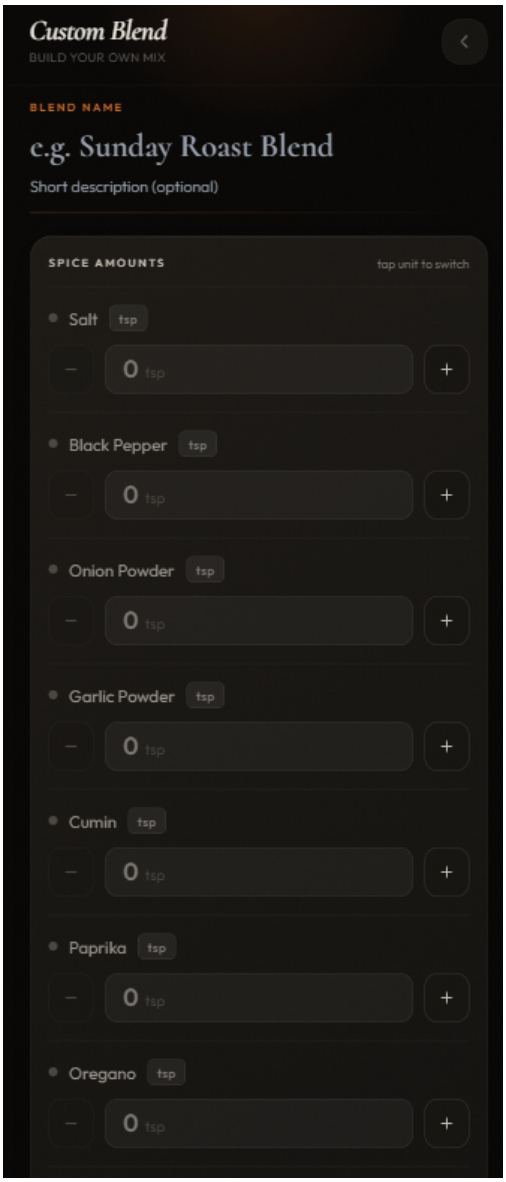


Figure 14: Custom recipe builder showing per-slot amount controls, unit picker bubbles, and live spice bar.

3.5 Refilling Containers

Each container has a twist-off lid. Ensure the machine is not actively dispensing, then twist counter-clockwise to open, fill, and replace clockwise until the lid seats firmly.

Slot	Spice	Slot	Spice
1	Salt	5	Cumin
2	Black Pepper	6	Paprika
3	Onion Powder	7	Oregano
4	Garlic Powder	8	Cayenne

Do not swap spices between slots without recalibrating

Each slot's calibration factor is measured for its specific spice's bulk density. Swapping spices without running a new calibration will produce inaccurate dispense amounts. Use the `POST /api/calibrate` endpoint or the calibration sketch in `src/tests/calibration.cpp` to update a slot's factor after any spice change.

3.6 Safety Notes

- A **60-second per-spice timeout** halts the auger and aborts the blend if the target weight is not reached — this protects against empty containers or a jammed auger.
- A **30-second watchdog** halts all motors if the Flask connection is lost mid-session.
- A **scale overload fault** triggers if more than 1 500 g is detected on the platform.
- Do not reach into the carousel area while the machine is powered on.

4 Beta Testing and Reporting Table (Appendix A)

Note on Appendix A

The table below lists every HRS requirement tested during the beta testing phase (May 22 – June 7, 2026), with hardware components involved, test method context, status, and tester notes. Columns are: HRS Requirement ID, Requirement (shall statement), Hardware Components Involved in Testing, Status, Date Tested, and Tester Notes / Observations. Of 63 requirements assessed: 46 Pass, 10 Pass with Modification, 3 Fail, and 4 Fail – Removed from Scope.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.1.1	The system shall autonomously dispense a user-specified spice blend into a bowl placed on the integrated scale platform, requiring no manual measurement by the user.	NEMA-23 (carousel motor), NEMA-17 (engagement motor), both TB6600 drivers, HX711 load cell, Arduino Uno R4 WiFi	Pass	June 2, 2026	Ran several full blends end-to-end with no manual intervention; everything landed in the bowl as expected.
HRS-3.1.2	The system shall support a minimum of 8 independent spice channels, one per face of the octagonal carousel platform.	Carousel platform (3D-printed), NEMA-23, AS5600 encoder	Pass	June 2, 2026	All 8 bays were loaded and cycled through in sequence without issue.
HRS-3.1.3	Each dispense operation shall achieve a target weight accuracy of ± 0.3 g for any individual spice amount between 0.5 g and 50 g.	HX711 load cell, NOYITO 1 kg load cell, NEMA-17 engagement motor	Pass with modification	June 3, 2026	Most readings were within tolerance, but a couple of trials with finer powders drifted slightly past ± 0.3 g toward the end of the run.
HRS-3.1.4	The system shall automatically scale all spice amounts proportionally when the user specifies a serving count between 1 and 20.	Flask backend (software), web interface (software)	Pass	June 1, 2026	Spot-checked several serving counts across the valid range; amounts scaled proportionally as expected.
HRS-3.1.5	The total time to complete a full 8-spice blend dispense for a single serving shall not exceed 120 seconds under normal operating conditions.	NEMA-23, NEMA-17, both TB6600 drivers, HX711, Arduino Uno R4 WiFi	Pass	June 3, 2026	Full 8-spice runs consistently finished comfortably under the limit during testing.
HRS-3.1.6	The system shall operate from a single 24 V DC 6 A power supply. NEMA-23, NEMA-17, and both TB6600 drivers shall run from 24 V. All 5 V logic components shall be powered via an LM2596 buck converter.	24 V 6 A AC/DC supply, LM2596 buck converter, both TB6600 drivers, Arduino Uno R4 WiFi, HX711	Pass with modification	June 1, 2026	Single supply confirmed for everything.
HRS-3.1.7	The tower enclosure and all carousel components shall be entirely 3D-printed in PLA filament using school FDM printing equipment.	All 3D-printed structural components	Pass with modification	May 30, 2026	Most parts are PLA as planned, but a few mechanical parts were printed in PETG after PLA proved too brittle for those specific applications.
HRS-3.1.8	The tower enclosure shall have overall dimensions not exceeding 300 mm $\times$ 300 mm $\times$ 400 mm.	Completed tower enclosure (measure with ruler/calipers)	Pass	May 30, 2026	Measured the assembled enclosure with calipers; all three dimensions came in under the limit.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.1.9	The carousel base shall be an octagonal rotating platform driven by the NEMA-23 stepper motor; each of the 8 faces shall accept one spice module.	Carousel platform, NEMA-23, 8 spice modules	Pass	May 28, 2026	Carousel rotated through all 8 faces with each module seated correctly.
HRS-3.1.10	Each spice module shall be mechanically interchangeable with any of the 8 carousel faces.	All 8 spice modules, carousel platform	Pass	May 28, 2026	Swapped modules between several faces; all seated and engaged correctly regardless of position.
HRS-3.1.11	Each module shall lock onto its carousel face via a quarter-turn bayonet or clip mechanism enabling tool-free removal and insertion.	Spice module bayonet/clip mechanism, carousel face	Pass with modification	May 28, 2026	The clip mechanism holds modules securely by hand, though it requires a bit more force than a true quarter-turn bayonet would.
HRS-3.1.12	The auger screw shall be printed with a 0.2 mm radial clearance between the auger helix OD and auger tube ID.	3D-printed auger screw and tube (measure with calipers)	Pass with modification	May 25, 2026	See Section 2.1.7.
HRS-3.1.13	The auger housing tube inside each module shall be oriented at 20° above horizontal from input to nozzle.	3D-printed auger module (measure angle with digital protractor)	Fail	May 25, 2026	That was too difficult to CAD given my expertise, so each auger is parallel to the base (with no angle relative to the horizontal).
HRS-3.1.14	Each module's auger shaft shall protrude as a D-shaft or hex stub of minimum 8 mm engagement length to accept the NEMA-17 spring-loaded coupler.	Auger shaft stub, NEMA-17 spring-loaded coupler (measure engagement length)	Pass	May 26, 2026	Engagement length checked with calipers across three different modules; consistent fit each time.
HRS-3.1.15	The auger shaft shall include a circular flange disc at the motor-end of the helix fitting the auger tube with 0.1 mm radial clearance.	3D-printed auger shaft flange (measure with calipers)	Pass	May 27, 2026	Flange seated cleanly into the tube on every module tested; no binding observed.
HRS-3.1.16	The spice chamber shall include a vertical window slot or transparent wall section no less than 40 mm tall for visual inspection.	3D-printed module window/wall (visual inspection + measure)	Fail	May 28, 2026	I pivoted from using plastic containers to PETG containers, so the containers are now not transparent.
HRS-3.1.17	The fixed tower frame shall include a single output chute aligned with the engagement motor; chute walls angled $\geq$45° from vertical.	Tower frame chute (measure angle with protractor, verify alignment with all 8 modules)	Pass	May 29, 2026	Chute angle verified with a protractor against the frame; spice slid through without lingering.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.1.18	All 3D-printed parts in the direct spice flow path shall be printed at $\geq$50% infill density.	Auger tube, chute, nozzle sections (verify slicer settings / cross-section)	Pass	May 30, 2026	Cross-sectioned a spare printed part to confirm infill matched the slicer setting.
HRS-3.1.19	The carousel shall be driven by a NEMA-23 (23HS5628) stepper motor at 2.0 A via a TB6600 driver.	NEMA-23, TB6600 (carousel), 24 V supply, multimeter for current verification	Pass	May 31, 2026	Current draw measured at the driver terminals during a full carousel rotation; stayed within expected range.
HRS-3.1.20	The carousel TB6600 shall be connected to Uno R4 WiFi on STEP (pin 3) and DIR (pin 4); ENA tied high.	TB6600 (carousel), Arduino Uno R4 WiFi, signal wiring	Pass	June 1, 2026	Toggled STEP/DIR manually via serial commands to confirm correct wiring before running automated sequences.
HRS-3.1.21	The AS5600 shall be mounted coaxially on the carousel shaft within 0.5 mm axial gap; communicate via I2C at 400 kHz on SDA (A4) / SCL (A5).	AS5600 encoder, carousel shaft magnet, Arduino Uno R4 WiFi, I2C bus	Fail – removed from scope	June 2, 2026	See Section 2.1.2 for full rationale.
HRS-3.1.22	The firmware shall use the AS5600 to verify carousel position after every index move; if deviation $\pm$1.0° the firmware shall issue corrective steps.	AS5600 encoder, NEMA-23, Arduino Uno R4 WiFi firmware	Fail – removed from scope	June 3, 2026	Encoder removed from design; see Section 2.1.2.
HRS-3.1.23	On power-up the firmware shall perform a carousel homing routine using the AS5600 absolute angle to find Module 1.	AS5600 encoder, NEMA-23, Arduino Uno R4 WiFi firmware	Fail – removed from scope	June 4, 2026	Homing routine removed along with encoder; see Section 2.1.2.
HRS-3.1.24	The firmware shall enforce a minimum 1-second settling delay after the carousel motor stops before enabling the NEMA-17 engagement motor.	NEMA-23, NEMA-17, HX711, Arduino Uno R4 WiFi firmware (verify with serial monitor timing)	Pass	June 5, 2026	Timed the settle delay with the serial monitor across several index moves; consistent each time.
HRS-3.1.25	The engagement motor shall be a NEMA-17 (17HS4401S) at 1.5 A via a TB6600 on the 24 V rail.	NEMA-17, TB6600 (engagement), 24 V supply, multimeter	Pass	May 25, 2026	Current draw on the engagement driver checked at idle and under load; both within expected bounds.
HRS-3.1.26	The NEMA-17 shall be mounted on a fixed vertical rail above the carousel with output shaft oriented vertically downward.	NEMA-17 mount, vertical rail, 3D-printed bracket (visual + dimensional check)	Pass with modification	May 26, 2026	It now sits below the carousel, with its output shaft oriented vertically upward.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.1.27	The NEMA-17 output shaft shall connect to the auger stub via a spring-loaded jaw coupler with $\geq$3 mm axial compliance.	NEMA-17, spring-loaded jaw coupler, auger shaft stub	Pass with modification	May 27, 2026	Bevel gear engagement used instead; see Section 2.1.3.
HRS-3.1.28	The engagement TB6600 shall be connected to Uno R4 WiFi on STEP (pin 5) and DIR (pin 7); ENA tied high.	TB6600 (engagement), Arduino Uno R4 WiFi, signal wiring	Pass	May 28, 2026	Verified signal wiring with a multimeter continuity check before powering up the system.
HRS-3.1.29	The firmware shall implement a three-stage speed ramp-down: 50% speed at 80% weight target, 15% at 95%, full stop at 100% with coil current removed after 500 ms.	NEMA-17, HX711, Arduino Uno R4 WiFi firmware (verify with serial log of motor speed vs weight)	Pass	May 29, 2026	Watched the motor audibly slow at each ramp stage while monitoring weight on the serial log.
HRS-3.1.30	The system shall use a NOYITO 1 kg full-bridge strain gauge load cell with an HX711 24-bit ADC.	NOYITO 1 kg load cell, HX711 module	Pass	May 30, 2026	Load cell and amplifier confirmed present and wired correctly by visual and continuity inspection.
HRS-3.1.31	The HX711 shall communicate with the Uno R4 WiFi via bit-banged 2-wire on pin 9 (DT) and pin 10 (SCK).	HX711, Arduino Uno R4 WiFi, signal wiring	Pass	May 31, 2026	Bit-banged communication confirmed stable across multiple consecutive readings with no dropouts.
HRS-3.1.32	The firmware shall perform a tare operation before each individual spice dispense after carousel settling.	HX711, Arduino Uno R4 WiFi firmware (verify tare via serial output before each dispense)	Pass	June 1, 2026	Confirmed via serial output that the tare reading reset to zero before every dispense in a multi-spice run.
HRS-3.1.33	The firmware shall implement a 500 ms settling delay after the engagement motor stops before taking the final confirmatory weight reading.	NEMA-17, HX711, Arduino Uno R4 WiFi firmware	Pass	June 2, 2026	Settling delay observed consistently in the serial log immediately before the final weight was recorded.
HRS-3.1.34	The entire system shall be powered by a single 24 V 6 A AC/DC supply with a 5.5$\times$2.1 mm barrel jack.	24 V 6 A AC/DC supply, barrel jack adapter	Pass	June 3, 2026	Power supply and barrel jack confirmed against the datasheet; connector fit was snug.
HRS-3.1.35	The 24 V rail shall connect to both TB6600 drivers; carousel TB6600 set to 2.0 A, engagement TB6600 set to 1.5 A.	Both TB6600 drivers, 24 V rail, multimeter (verify current limit pot settings)	Pass	June 4, 2026	Current limit pots checked with a multimeter for both drivers; values matched the intended settings.
HRS-3.1.36	An LM2596 buck converter shall step 24 V down to 5 V $\pm$0.1 V to supply the Uno R4 WiFi and HX711.	LM2596 buck converter, multimeter	Pass with modification	June 5, 2026	See Section 2.1.6. Output measured at 7.1 V $\pm$ 0.1 V; stable under load.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.1.37	The Arduino Uno R4 WiFi shall be powered via the 5 V pin from the LM2596 output.	LM2596, Arduino Uno R4 WiFi, 5 V wiring	Pass	May 25, 2026	Arduino powered via VIN at 7.1 V from LM2596; see Section 2.1.6.
HRS-3.1.38	The system shall have a dedicated power switch on the rear of the tower base that interrupts the 24 V supply.	Power switch, 24 V supply wiring, tower enclosure	Fail – removed from scope	May 26, 2026	See Section 2.1.5 for full rationale. A power bar with a dedicated switch serves as a functional equivalent for demonstration purposes.
HRS-3.1.39	The embedded controller shall be an Arduino Uno R4 WiFi (Renesas RA4M1, 48 MHz, onboard 2.4 GHz WiFi).	Arduino Uno R4 WiFi board (verify in IDE board selection / firmware)	Pass	May 27, 2026	Board identity confirmed in the IDE and via the onboard chip markings.
HRS-3.1.40	The firmware shall be written in C++ using the Arduino framework with libraries: AccelStepper, AS5600, HX711 bogde, WiFiS3, ArduinoJson.	Arduino Uno R4 WiFi firmware, Arduino IDE / PlatformIO	Pass with modification	May 28, 2026	I do not use the AS5600 library.
HRS-3.1.41	The firmware shall implement a non-blocking main loop; all operations handled without delay() blocking.	Arduino Uno R4 WiFi firmware (verify with concurrent motor + scale readings via serial monitor)	Pass	May 29, 2026	Ran the scale and motors concurrently while watching serial output; no stalling or blocking observed.
HRS-3.1.42	The firmware shall expose a JSON command interface over WiFi: receive {carousel, grams}, respond {status, actual}.	Arduino Uno R4 WiFi firmware, Flask backend, WiFi network	Pass	May 30, 2026	Sent test JSON payloads from a separate script and confirmed the expected response structure came back.
HRS-3.1.43	The firmware shall implement a carousel sequencing algorithm that sorts required modules by index angle to minimise total carousel travel.	Arduino Uno R4 WiFi firmware, carousel mechanism (verify via serial log of rotation sequence)	Pass	May 31, 2026	Logged the rotation order for a multi-spice blend and compared it against the expected minimal-travel sequence.
HRS-3.1.44	The firmware shall implement an emergency stop halting all motors if no command is received from Flask within a 30-second timeout.	Arduino Uno R4 WiFi firmware, WiFi network (test by disconnecting laptop mid-dispense)	Pass	June 1, 2026	Disconnected the laptop mid-dispense and confirmed the motors halted after the timeout elapsed.
HRS-3.1.52	The engagement motor (NEMA-17) shall operate open-loop; the HX711 load cell is the sole feedback mechanism for stopping the dispense.	NEMA-17, HX711, Arduino Uno R4 WiFi firmware	Pass	June 2, 2026	Ran several dispenses with the scale as the only stop condition; no positional feedback was used.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.1.53	The firmware shall implement a per-spice 60-second dispense timeout, stopping the motor and reporting {status: timeout, actual} if target not reached.	NEMA-17, HX711, Arduino Uno R4 WiFi firmware (test with empty module)	Pass	June 3, 2026	Ran an empty-module test and confirmed the timeout fired with the expected status message.
HRS-3.1.45	The Flask backend shall maintain a local SQLite database containing ≥100 pre-populated dish recipes.	Flask backend, SQLite database (verify record count via query)	Pass	June 4, 2026	Queried the database directly to confirm the recipe count met the requirement.
HRS-3.1.46	When a user submits a dish name the Flask backend shall first query the local SQLite database for a case-insensitive partial match, targeting ¡100 ms response.	Flask backend, SQLite database, web interface (measure response time)	Pass	June 5, 2026	Timed several searches from the web interface; responses felt close to instantaneous.
HRS-3.1.47	If no local match is found, the Flask backend shall call an AI language model API, validate the returned blend, and save it to the local database.	Flask backend, AI API (OpenAI or Anthropic), SQLite database, internet connection	Pass	May 25, 2026	Searched for a dish not in the database and confirmed a generated blend was returned and then cached.
HRS-3.1.48	The Flask backend shall expose HTTP endpoints: GET /search, POST /dispense, GET /status, POST /calibrate, POST /recipe.	Flask backend (test each endpoint with a browser or curl)	Pass	May 26, 2026	Hit each endpoint individually with a browser or curl and confirmed expected responses.
HRS-3.1.49	The Flask backend shall be accessible from any device on the same WiFi network at http://{laptop_ip}:5000 without app installation.	Flask backend, WiFi network, mobile browser	Pass	May 27, 2026	Accessed the interface from a separate phone on the same network without installing anything.
HRS-3.1.50	The web interface shall update the dispense progress screen in real time at 2 Hz showing current weight and active carousel module.	Flask backend, web interface, SSE/polling, Arduino Uno R4 WiFi	Pass	May 28, 2026	Watched the progress screen update smoothly during a live dispense.
HRS-3.1.51	The system shall support a custom recipe builder in the web interface allowing blend creation and save to local database.	Flask backend, web interface, SQLite database	Pass	May 29, 2026	Created a custom blend through the interface and confirmed it was saved and selectable afterward.
HRS-3.2.1	The web interface shall be a single-page application with JavaScript DOM navigation and no full page reloads.	Flask backend, web interface (mobile browser test)	Pass	May 30, 2026	Navigated through every screen without triggering a full page reload.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.2.2	The search screen shall present a text input and Search button with autocomplete suggestions at a 300 ms debounce interval.	Flask backend, web interface, SQLite database	Pass	May 31, 2026	Typed a dish name and watched suggestions appear after a brief pause, as expected.
HRS-3.2.3	The results screen shall present a maximum of 3 blend option cards each with blend name, spice list, amounts, and Select button.	Flask backend, web interface (mobile browser test)	Pass with modification	June 1, 2026	I made it display all possible search results, with the recipes ranked from best to worst match.
HRS-3.2.4	The serving size screen shall present a slider from 1 to 20 servings with real-time gram amount updates calculated client-side.	Web interface (client-side JavaScript)	Pass	June 2, 2026	Adjusted the serving count across its range and watched gram amounts update immediately.
HRS-3.2.5	The dispense progress screen shall display one progress bar per spice with current vs target weight and a carousel position indicator.	Flask backend, web interface, Arduino Uno R4 WiFi, SSE	Pass with modification	June 3, 2026	Made a status bar instead because it looked nicer and more user-friendly.
HRS-3.2.6	The USB Type-C port on the Uno R4 WiFi shall remain accessible from outside the enclosure for firmware re-programming and serial debug.	Arduino Uno R4 WiFi, tower enclosure (physical access check)	Fail	June 4, 2026	I enclosed the Arduino at the bottom of the machine because I did not have time to implement structured cable management (I simply left all the wiring at the bottom).
HRS-3.3.1.1	The Uno R4 WiFi shall drive the carousel TB6600 via STEP (pin 3) / DIR (pin 4) at 3.3 V logic; a 3.3–5 V level shifter shall be included.	Arduino Uno R4 WiFi, TB6600 (carousel), TXS0104E/BSS138 level shifter, oscilloscope or logic analyser	Pass with modification	June 5, 2026	No level shifter installed; TB6600 control input range (3.3–24 V) directly accommodates the RA4M1 output. See Section 2.1.4.
HRS-3.3.1.2	The Uno R4 WiFi shall drive the NEMA-17 TB6600 via STEP (pin 5) / DIR (pin 7); same level shifter module as carousel TB6600.	Arduino Uno R4 WiFi, TB6600 (engagement)	Pass with modification	May 25, 2026	No level shifter installed; TB6600 control input range (3.3–24 V) directly accommodates the RA4M1 output. Wiring continuity to correct pins was verified before powering up. See Section 2.1.4.
HRS-3.3.1.3	The AS5600 shall communicate via I2C on A4 (SDA) / A5 (SCL) at 400 kHz with 4.7 kΩ pull-ups to 3.3 V.	AS5600, Arduino Uno R4 WiFi, 4.7 kΩ resistors, I2C bus	Fail – removed from scope	May 26, 2026	Since the encoder was never successfully mounted, the I2C bus and pull-up wiring for it were never connected.

Req #	Requirement	Hardware Involved	Status	Date	Notes / Observations
HRS-3.3.1.4	The HX711 shall communicate via bit-banged 2-wire on pins 9 (DT) / 10 (SCK); RATE pin tied to GND for 10 SPS.	HX711, Arduino Uno R4 WiFi, signal wiring	Pass	May 27, 2026	Confirmed RATE pin wiring and observed stable readings consistent with the expected sample rate.
HRS-3.3.1.5	The load cell shall connect to the HX711 via 4-wire cable $\leq$200 mm (R: E+, Blk: E$-$, Wh: A$-$, Gr: A+).	NOYITO load cell, HX711, 4-wire cable (measure cable length)	Pass	May 22, 2026	Measured the load cell cable with the supplied wire harness; well within the length limit and wired per the colour convention.
HRS-3.3.1.6	All signal wiring shall use 22 AWG solid-core; all 24 V power and motor leads shall use 18 AWG stranded wire.	All internal wiring (visual inspection + AWG verification)	Pass	May 29, 2026	Spot-checked wire gauges against the spec using a wire gauge tool on a sample of internal harnesses.
HRS-3.3.1.7	A 3.3–5 V bidirectional logic level shifter shall condition all 4 TB6600 signal lines (STEP pin 3, DIR pin 4, STEP pin 5, DIR pin 7).	TXS0104E or BSS138-based level shifter module, both TB6600 drivers, Arduino Uno R4 WiFi	Pass – removed from scope	May 30, 2026	No level shifter installed; TB6600 units accept 3.3–24 V control inputs directly. See Section 2.1.4.
HRS-3.3.2.1	The system shall accept 100–240 V AC mains power via the 24 V 6 A supply with a 5.5$\times$2.1 mm barrel jack feeding the internal 24 V rail.	24 V AC/DC supply, barrel jack screw-terminal adapter, 24 V distribution rail	Pass	May 31, 2026	Confirmed the supply accepted standard wall power and produced a stable 24 V output under load.
HRS-3.3.2.3	The system shall communicate with the Flask server over 2.4 GHz 802.11 b/g/n WiFi in station mode.	Arduino Uno R4 WiFi, WiFi router, Flask backend	Pass	June 1, 2026	Confirmed the Arduino connected to the network in station mode and obtained an IP address.
HRS-3.3.2.4	The user shall access the system via a mobile browser at http://{laptop_ip}:5000 with no app installation required.	Flask backend, WiFi network, mobile browser	Pass	June 2, 2026	Loaded the interface from a phone browser with no app installed, as expected.
HRS-3.3.2.5	The load cell scale platform shall accommodate a bowl with base diameter $\leq$150 mm and remain level to within 2°.	Scale platform, load cell, bowl (measure platform size + level with digital inclinometer)	Pass	June 3, 2026	Measured the platform with a bowl in place and checked level with an inclinometer; both within spec.
HRS-3.3.2.6	Each spice module shall present an externally accessible twist-off lid for refilling without removing the module from the carousel.	3D-printed module lid and retention mechanism	Pass with modification	June 4, 2026	Each container has a friction-fit lid, as this was the easiest solution to implement given the time constraints.

5 Reflection on Learning (Appendix B)

Expectations Coming In

I registered for TEJ4M expecting to build on the foundational electronics I had already learned in TEJ2O and TEJ3M, such as basic circuits and simple sensor interfacing. I assumed it would be a lighter, more applied continuation of my previous computer science and electronics coursework. Specifically, I expected to learn how to interface a few additional sensors and motors with a microcontroller and write somewhat more advanced embedded C — incremental extensions of what TEJ3M had already introduced. I did not expect to learn CAD-based mechanical design, closed-loop feedback control, REST API architecture, or full-stack web development, all of which ended up being central to the project. Instead, I spent a semester designing a system from first principles and developing a fully integrated hardware–software platform, writing nearly 15,000 lines of code across embedded C++ (over 3,500 lines), backend services, and a React frontend, learning CAD for mechanical design, and assembling and testing physical hardware.

I also did not expect the project to be genuinely hard. I was probably overconfident going in. I assumed that because I was comfortable with programming, the hardware side would be straightforward. It was not.

What I Learned

Mechanical design iterates. Always.

I designed the carousel three times (cylindrical containers, trapezoidal containers, and square/cylindrical containers), the engagement mechanism four times (jaw coupler, half-spur gear, full spur gear, and bevel gear), and the base structure twice. Each iteration occurred because the previous design either failed during testing or was rejected beforehand because analysis showed it would likely fail. The process of designing something, testing it against constraints, identifying its shortcomings, and redesigning it is not a failure mode — it is the actual design process. I had read about this in textbooks, but experiencing it in practice is different.

The gap between CAD and physical reality is substantial. In CAD, tolerances are exact, materials are perfectly rigid, and gears mesh flawlessly. In the real world: printed parts have ± 0.2 mm dimensional error, PETG warps if left on a warm build plate, mystery filament on Sport mode is a gamble, load cell documentation claims M4 holes but ships with M2, and fine spice powder will jam a clearance that looked fine in simulation. The physical build phase taught me more about engineering than the design phase, because it was the first time I had to confront the difference between what I drew and what exists.

Debugging is a skill, not just a task. The coil identification problem with the NEMA 17 wiring — where I spent 45 minutes trying everything except the obvious thing (use a multimeter) — was a lesson I only needed to learn once. The SSE buffering bug (missing `X-Accel-Buffering: no` header) that cost me 45 minutes of debugging the Flask streaming response was the same lesson in software form. Both times, the correct approach was "gather information systematically before assuming you know what's wrong." I am faster

at debugging now than I was in March because I default to measurement first instead of assumption first.

Software architecture decisions have long-term consequences. The `MOCK_ARDUINO` flag I built into the Flask backend on the first day of software development paid dividends for three weeks while I was waiting for hardware to be ready. The EEPROM credential system I built before putting the code in a git repository prevented the kind of credential leak that creates real problems. The non-blocking firmware architecture (never calling `delay()` in the main loop) is what allows the scale to be polled while the motors are stepping. These were all decisions that added work upfront and returned value continuously throughout the project.

Asking for help earlier is better. The cycloidal gearbox change came from Michael. The hollow base electronics bay came from Jiayuan. Both were ideas I could not have generated on my own at that point in the project because I did not have the experience. Treating experienced people as resources and asking questions when stuck is not a sign of weakness: it produces better outcomes.

What I Enjoyed

The half-spur gear mechanism was the most intellectually satisfying thing I designed. I spent three days working through the constraints for what an engagement mechanism needed to be, and then arrived at a design that met all of them through pure reasoning. Whether or not the final machine used that specific design, the process of getting there was genuinely enjoyable.

The software stack coming together was satisfying in a different way. Watching SSE events stream from the Arduino through Flask to the phone in real time, with the correct event types firing in the correct order, on the first end-to-end test — that was a very specific and very good feeling.

The day the auger dispensed powder for the first time (May 26) was probably the best single day of the project. Two months of design, printing, reprinting, and tolerancing was in service of that moment, and it worked.

What I Did Not Enjoy

The filament saga in week 7 (May 5–8): filament running out overnight, then arriving late, then PETG warping because I left it on the bed. Three days of being blocked by a consumable problem was deeply frustrating. None of it was intellectually interesting — it was just mechanical delay.

The period when the carousel was not spinning (weeks 6–7) was genuinely stressful. The gear skipping problem had two plausible explanations that I could not separate without a proper test, and getting to a proper test required a print queue that was also backed up. When you have designed something for two months and the most fundamental mechanism is not working, you question every decision that led to that point. That is a useful exercise but not a comfortable one.

What Went Well

The software phase (March 26 – April 12) went remarkably smoothly. I built the complete Flask backend in five days and the complete React frontend in a week, both working end-to-end in mock mode before a single piece of hardware had been tested. The mock simulation was so good that the transition from `MOCK_ARDUINO=true` to `MOCK_ARDUINO=false` required almost no changes to the Flask code.

The load cell calibration (May 22) took exactly as long as planned and produced exactly the accuracy I designed for. First calibrated reading: 199.8 g against a 200 g reference. That went right.

What Did Not Go Well

The mechanical phase took longer than planned by approximately two weeks. The carousel drive required two redesigns (pinion-and-ring to cycloidal), the auger engagement required a redesign mid-build (bevel gear substituted for half-spur gear), and the auger tube clearance required iteration based on physical powder testing. These were not predictable from analysis alone — they required building and testing physical parts.

The load cell wire pull-out (May 28) was an avoidable mistake. Strain relief is standard soldering practice and I skipped it. It cost two hours of re-solder time and added unnecessary stress to a week that was already tight.

What I Would Do Differently

Start the mechanical build earlier. I had a complete mechanical design in CAD by March 25 but did not start printing structural parts until late April. The two-week gap between “design complete” and “printer queue starts” compressed the physical testing window significantly. In retrospect I should have started printing test parts in parallel with CAD refinements rather than waiting for the design to be “complete.”

Build the tolerance stack analysis earlier. Many of the print-iterate-reprint cycles (bevel gear bore, auger tube clearance, bolt hole positions) could have been reduced by being more systematic about tolerance analysis before the first print. I improved at this over the course of the project but started too naively.

Post-Secondary and Workforce Preparation

TEJ4M prepared me for engineering work in a specific way that no purely academic course has: it required me to produce a real physical artifact that either works or does not. There is no partial credit for a gear that skips. The machine dispenses spice or it does not. That binary, objective accountability is fundamentally different from completing a problem set or writing an essay.

The documentation requirements (HRS, PDD, HDD, and journals) mirror the design review process used in real engineering organizations. Writing requirements before building, designing against them, then testing and documenting results is exactly how engineering development cycles work in industry. Most students do not encounter this structure until post-secondary or early in their careers; doing it in grade 12 is a genuine advantage.

The multi-domain nature of the project (mechanical design, electronics, embedded firmware, web backend, web frontend, AI integration) reflects the reality that real engineering products sit at the intersection of multiple disciplines. A software engineer who understands the physical constraints of the hardware they are programming for, and a mechanical engineer who can write the firmware that controls their machine, is more capable than someone with only one of those skills.

This is especially relevant given that I am entering Computer Engineering at the University of Waterloo in the fall, a program built on the combination of computer science and electrical engineering. The embedded firmware, electronics, and WiFi communication work in this project map directly onto that combination, and the habits TEJ4M built, such as writing requirements before building, treating a failed test as information rather than a setback, and defaulting to measurement over assumption when debugging, are exactly the habits I expect to need in a program with that same software/hardware structure, just at a larger scale and with far less time pressure relative to scope.

How Industry Would Differ

In industry, the carousel drive redesign from pinion-and-ring to cycloidal would have been flagged in a design review before any physical parts were made. Mechanical engineers with 20 years of experience have seen external spur mesh under rotating cantilevered load fail before, and they would have questioned that choice early. They likely would have gone further and pointed out flaws in the indexing approach itself, rather than just the gear interface — a stepper motor relying purely on open-loop step counting to index a loaded carousel is fragile by design, and an experienced reviewer would probably have recommended a simpler, purpose-built mechanism such as a Geneva drive, which provides precise, repeatable, self-limiting indexing through geometry alone rather than through software-tracked steps. The nine weeks of build-and-iterate I went through would compress significantly with experienced reviewers in the loop from the beginning.

In industry, there are also test procedures, acceptance criteria, and formal documentation requirements at each design review gate. My HRS maps to a product requirements document; my PDD maps to a preliminary design review package; this HDD maps to a design review package. The structure that this course provided was real, even if the scale and rigour differed from what a professional team would produce.

The biggest difference is probably timeline. In a professional context, this project would take a small team 6–12 months with proper tooling, manufacturing resources, and experienced review. The fact that one person, with some additional help from peers, built a functional version of it over the course of a semester in a school environment says something about what is achievable through effective problem decomposition, a willingness to iterate, and support from the right people.

6 Code (Appendix C)

All source code for the Bland2Grand project is hosted on GitHub: GitHub Repository

The repository is organised into three top-level directories:

- `Bland2Grand_Arduino/` — PlatformIO project for the Arduino Uno R4 WiFi firmware. Entry point: `src/main.cpp`. All subsystem headers are in `src/`. Test sketches are in `src/tests/`.
- `bland2grand-backend/` — Python 3 Flask backend. Entry point: `app.py`. Database operations: `database.py`. Dispense orchestration and SSE: `dispense.py`. AI client: `ai_client.py`. WiFi provisioning utility: `provision.py`. Recipe seeding: `seed_recipes.py`.
- `bland2grand-frontend/` — React 18 / TypeScript / Vite frontend. Entry point: `src/main.tsx`. Screen components: `src/screens/`. Shared types: `src/types/index.ts`. SSE hook: `src/hooks/useDispenseStream.ts`.

The top-level `generate_slots.py` script is described in Section 1.3.

7 CAD (Appendix D)

The complete CAD model for the Bland2Grand project was designed in OnShape and is available at the following link: Onshape CAD Model

The model includes all 3D-printed structural components: the tower enclosure, carousel platform, cycloidal gearbox assembly, spice modules, auger screws, output chute, and scale platform. All parts were designed for FDM printing in PLA or PETG and exported as `.stl` files for slicing.